Manual de Estruturação: Agentic OS (Sistema Operacional Agêntico)

Este documento define a especificação técnica para a implementação de um **Agentic OS**, uma arquitetura modular de diretórios e arquivos projetada para otimizar a interação com **Sistemas Agênticos** (como Claude Code ou Codex). O objetivo é transcender as limitações de sessões de chat efêmeras, estabelecendo uma estrutura de memória persistente e execução especializada que elimina a redundância de contexto.

1. Introdução e Visão Geral do Sistema

O **Agentic OS** não é um software, mas uma metodologia de organização de arquivos que permite a um agente de IA operar com autonomia e precisão dentro de um ambiente de arquivos local. Ao estruturar o conhecimento e as regras em diretórios específicos, o arquiteto de soluções garante que o agente possua um "Saber de Negócio" pronto para uso.**Benefícios Estruturais:**

- **Prontidão de Cenário (Context-Ready):** Elimina a necessidade de fornecer prompts de contexto extensos a cada nova interação.
- **Token Efficiency (Eficiência de Execução):** Otimiza o consumo de tokens ao permitir que o agente carregue apenas os módulos necessários para a tarefa atual.
- **Evolução Sistêmica:** Através de uma camada de memória persistente, o sistema evolui continuamente, consolidando aprendizados entre diferentes sessões.

2. Camada 1: Identidade (O Orquestrador Central)

O arquivo raiz atua como o ponto de entrada (entry point) do sistema, definindo a personalidade e as diretrizes mestras do agente.

- **Nomenclatura Técnica:** Deve ser nomeado claud.md para implementações no Claude Code ou ai.md para Codex e outros sistemas similares.
- **Responsabilidades:** Detalha o perfil do usuário (ex: Lucas, criador de conteúdo), audiência-alvo, regras de comunicação, tons preferenciais e restrições operacionais críticas.
- **Otimização do Context Window:** Para evitar a "inflação de contexto" — onde a IA começa a ignorar instruções devido ao excesso de tokens — este arquivo deve ser limitado a **100-200 linhas**. Informações mais densas devem ser segregadas na pasta /context e referenciadas no orquestrador, mantendo o núcleo do sistema leve e responsivo.

3. Camada 2: Conhecimento (Pasta /context)

Esta camada armazena o "Saber do Negócio" de forma modular. Em vez de um documento único, o conhecimento é fragmentado para facilitar a recuperação precisa de informações.**Módulo de Exemplo: Estratégia de Conteúdo (Canal de Tecnologia)**| Arquivo | Conteúdo Esperado (Standardized Schema) || ------ | ------ || canal.md | Definição do nicho, objetivos de crescimento e proposta de valor da plataforma. || produtos.md | Especificações técnicas de ofertas, serviços ou infoprodutos vinculados ao sistema. || metricas.md | Benchmarks de performance, KPIs e análise de projetos/vídeos de maior sucesso. || tom_de_voz.md | Diretrizes semânticas e exemplos de escrita para garantir a originalidade da marca. |

4. Camada 3: Memória e Aprendizado (Pasta /memory)

A camada de memória transforma o agente em um sistema de aprendizado contínuo, funcionando como um banco de dados de interações passadas para evitar o problema do "cold start" (iniciar do zero).

- **Subpasta /learnings :** Repositório de insights, ajustes de estratégia e descobertas extraídas de tarefas concluídas.
- **Subpasta /history :** Logs estruturados das sessões para auditoria e recuperação de contexto histórico.
- **Consolidation Triggers (Hooks e Wrapup):** O sistema utiliza **Hooks** (gatilhos automáticos após tarefas) e o comando /wrapup para forçar a IA a consolidar a sessão atual. Este processo extrai novos aprendizados e atualiza os arquivos de memória antes do encerramento do terminal, garantindo a persistência dos dados.

5. Camada 4: Trabalhadores e Subagentes (Pasta /workers)

Dentro da arquitetura modular, os *Workers* são arquivos que definem especialistas para funções específicas, permitindo que o Agentic OS delegue tarefas complexas a subagentes com foco restrito.**Exemplos de Especialistas:**

- **Roteirista:** Focado em estrutura narrativa, retenção e ganchos (hooks) de engajamento.
- **Analista:** Especializado em extração de padrões e revisão de métricas técnicas.
- **Pesquisador:** Atua na busca de referências externas e validação de fatos técnicos.Cada arquivo Markdown em /workers deve obrigatoriamente declarar o **Papel (Role)** , a **Função Operacional** e o **Esquema de Saída esperado** .

6. Camada 5: Automações, Workflows e Verificações

Esta camada gerencia a padronização e a segurança operacional do sistema.

- **Slash Commands:** Comandos de atalho (ex: /novo_roteiro) que orquestram fluxos de trabalho entre múltiplos subagentes de forma automatizada.
- **Evaluation Layer (evaluation.json):** Define critérios de sucesso e esquemas de dados fixos. Isso garante a **Consistência de Output** ; por exemplo, exigindo que todo roteiro siga um JSON com os campos "Nome" e "Descrição", evitando variações indesejadas no formato de entrega.
- **Guardrails:** Atuam como buffers de segurança e auditoria, definindo permissões rigorosas para chamadas de ferramentas externas e exigindo confirmação humana para ações de alto impacto (ex: deleção de arquivos ou acesso a APIs pagas).

7. Blueprint da Hierarquia de Diretórios

Abaixo, a representação visual da arquitetura técnica recomendada:

```
/
├── claud.md (ou ai.md)      # Orquestrador Central (Identidade e
Regras)
├── /context/               # Base de Conhecimento Modular
│   ├── canal.md
│   ├── produtos.md
│   └── tom_de_voz.md
├── /memory/                # Persistência e Aprendizado
```

```
│   ├── /learnings/
│   └── /history/
├── /workers/              # Subagentes Especializados
│   ├── roteirista.md
│   ├── pesquisador.md
│   └── analista.md
├── /automation/           # Governança e Validação
│   ├── evaluation.json    # Standardized Output Schema
│   └── guardrails.md      # Safety Buffers
└── /projects/             # Depósito de Entregáveis Finais
```

8. Guia de Implementação via Master Prompt

Utilize o prompt abaixo para inicializar o Agentic OS em um novo diretório. Este prompt foi desenhado para garantir integridade estrutural e referenciamento cruzado.**Prompt Mestre: Arquitetura de Agentic OS**"Aja como um Senior AI Solutions Architect. Seu objetivo é inicializar um Agentic OS seguindo uma arquitetura modular e persistente:

1. **Fase de Descoberta:** Questione-me sobre o nicho de atuação, objetivos de negócio, audiência e tom de voz para alimentar a camada de conhecimento.
2. **Setup Estrutural:** Crie a hierarquia física de pastas: /context, /memory/learnings, /memory/history, /workers, /automation e /projects.
3. **Configuração do Orquestrador:** Gere o arquivo raiz claud.md (ou ai.md) detalhando minha identidade e restrições operacionais. Mantenha o arquivo entre 100-200 linhas para otimização da Context Window, utilizando referências cruzadas para a pasta /context.
4. **Desenvolvimento de Conhecimento:** Popule a pasta /context com arquivos Markdown baseados na nossa fase de descoberta, garantindo separação modular dos temas.
5. **Deploy de Workers:** Configure os subagentes necessários em /workers, definindo Papel, Função e Formato de Saída (Standardized Schema).
6. **Persistência de Memória:** Implemente o comando /wrapup e configure a lógica de hooks para garantir que todo aprendizado de sessão seja automaticamente consolidado em /memory/learnings para persistência entre execuções."